SIMATS ENGINEERING

CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

Register Number	**192412015**
Name	**K.S.Dharshana**

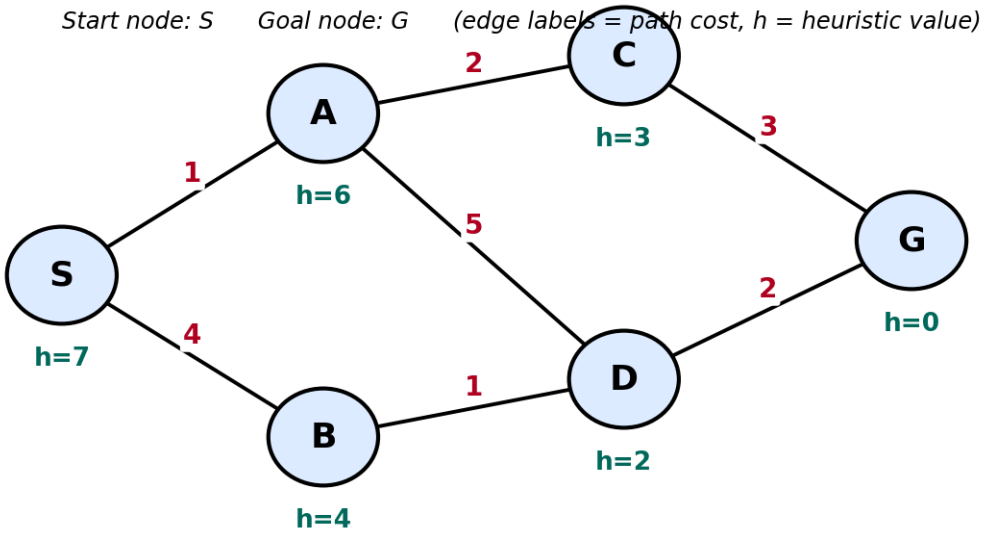
COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

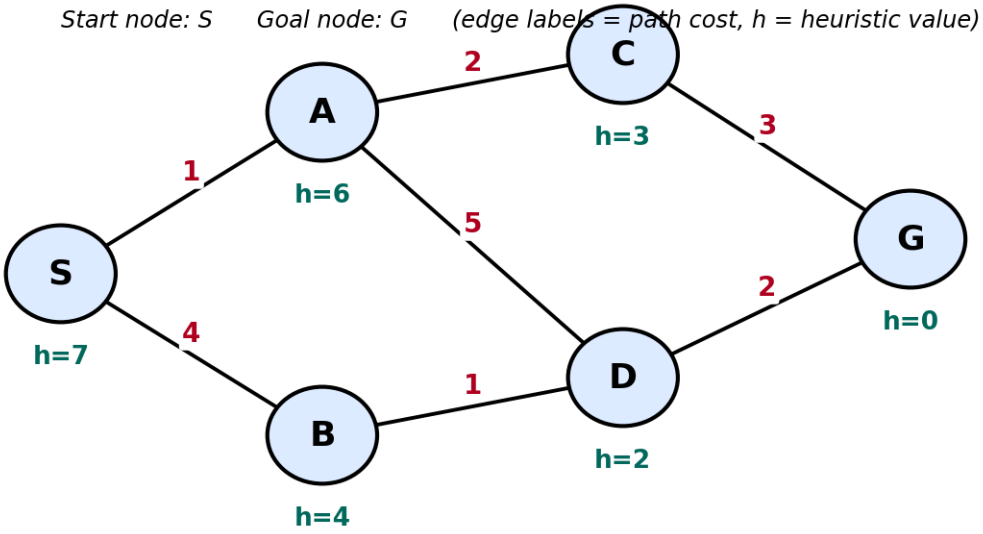
QUESTIONS: 5 **Total Marks:50**

Q1. Consider a suitable state-space graph (with heuristic values h(n) assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.

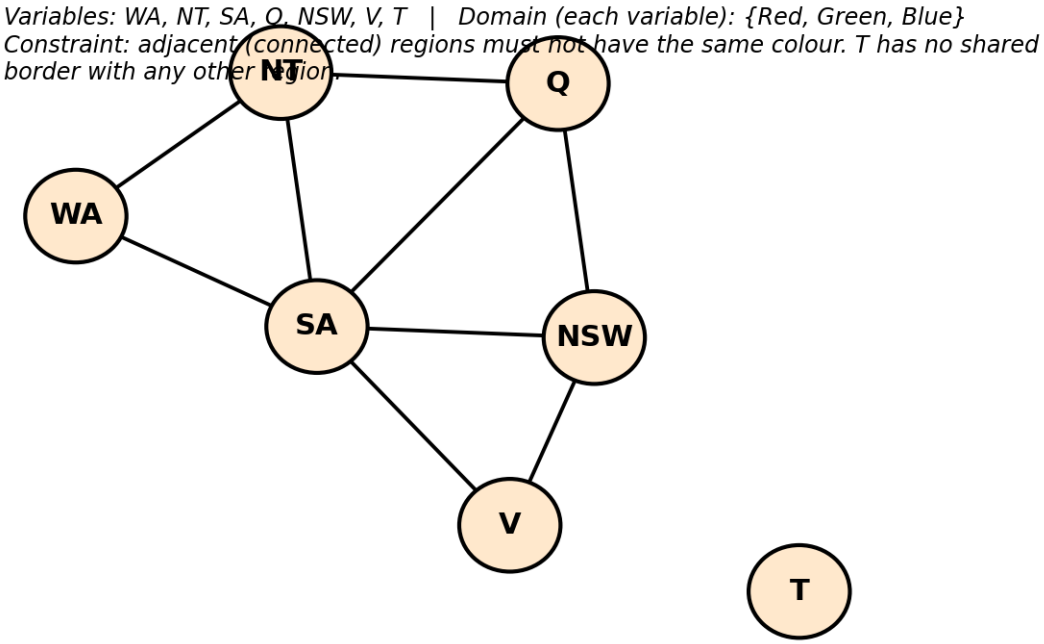


Q2. For a suitable weighted state-space graph (with edge costs and heuristic values h(n)) of your choice, perform a dry run of A* Search from the start node to the goal

node. Show the g(n), h(n) and f(n) values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.

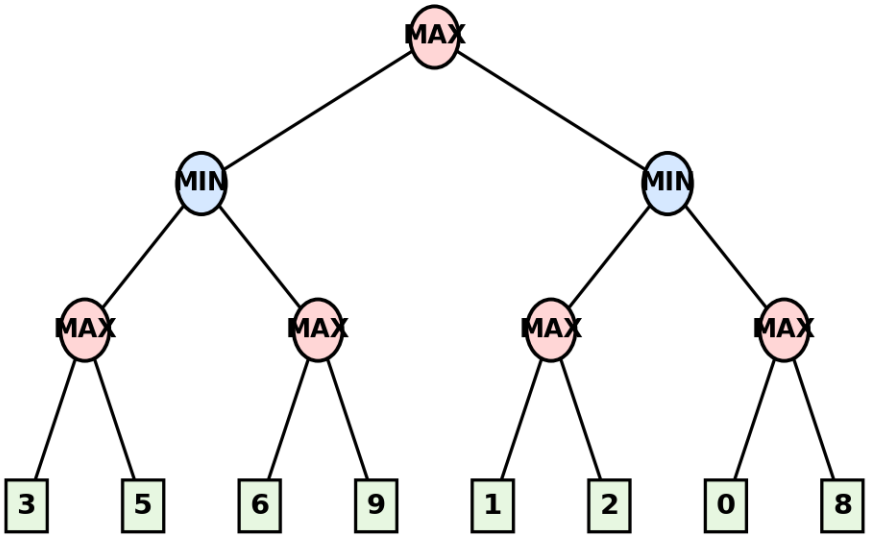


Q3. Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.



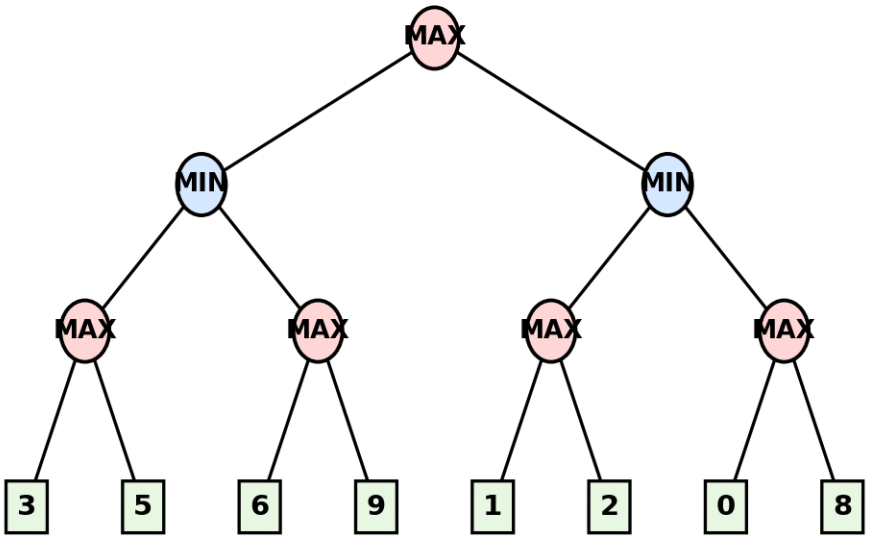
Q4. For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



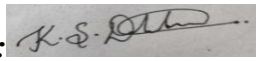
Q5. Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Code of Conduct:

I K.S.Dharshana (192412015) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Dry Run Evaluation

RUBRICS FOR CALCULATION

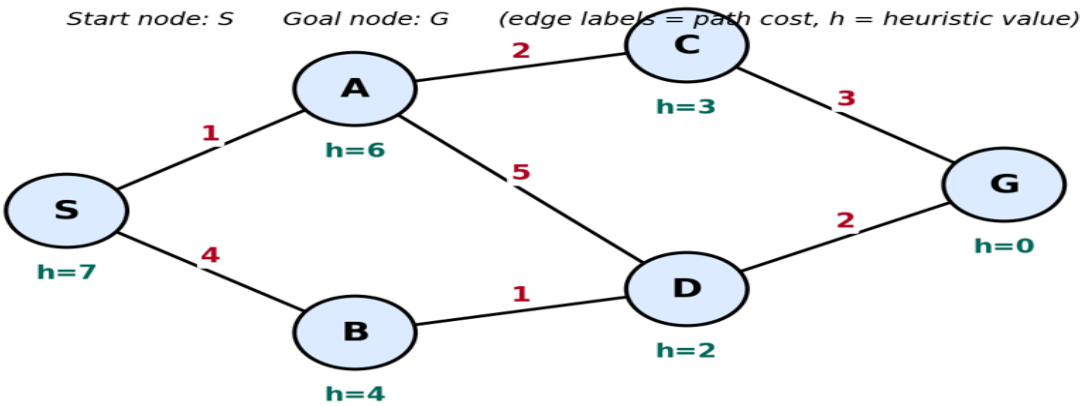
Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	**10**
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	**10**
Accuracy of Computations (g(n), h(n), f(n) values, minimax backed-up values, or α–β updates)	All computed values are accurate and consistent throughout the dry run.	Minor calculation errors are present but do not significantly affect the	Multiple calculation errors are present that affect the correctness of	Calculations are mostly incorrect or missing.	**10**

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
		final outcome.	intermediate steps.		
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	**10**
Clarity of Presentation & Justification (trace tables/trees/ diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation.	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	**10**
				Total	50

Q1. Greedy Best-First Search

Problem Statement

Consider the given state-space graph with heuristic values. Perform a dry run of the Greedy Best-First Search algorithm from the start node **S** to the goal node **G**. Show the frontier (Open List), node expansion order, and the final path obtained.



Algorithm (Greedy Best-First Search)

1. Start from the initial node.
2. Add the start node to the Open List.
3. Select the node having the smallest heuristic value **h(n)**.
4. Expand the selected node and insert its successors into the Open List.
5. Repeat until the goal node is reached.
6. Return the path from the start node to the goal node.

Dry Run

Step 1

Open List:

NODE	h(n)
S	7

Expand **S**

Generated successors:

- A (h = 6)
- B (h = 4)

Updated Open List:

NODE	h(n)
B	4
A	6

Step 2

Expand **B** because it has the lowest heuristic value.

Generated successor:

- D (h = 2)

Updated Open List:

NODE	h(n)
D	2
A	6

Step 3

Expand **D**

Generated successor:

- G (h = 0)

Updated Open List:

NODE	h(n)
G	0
A	6

Step 4

Expand **G**

Goal node is reached.

Expansion Order

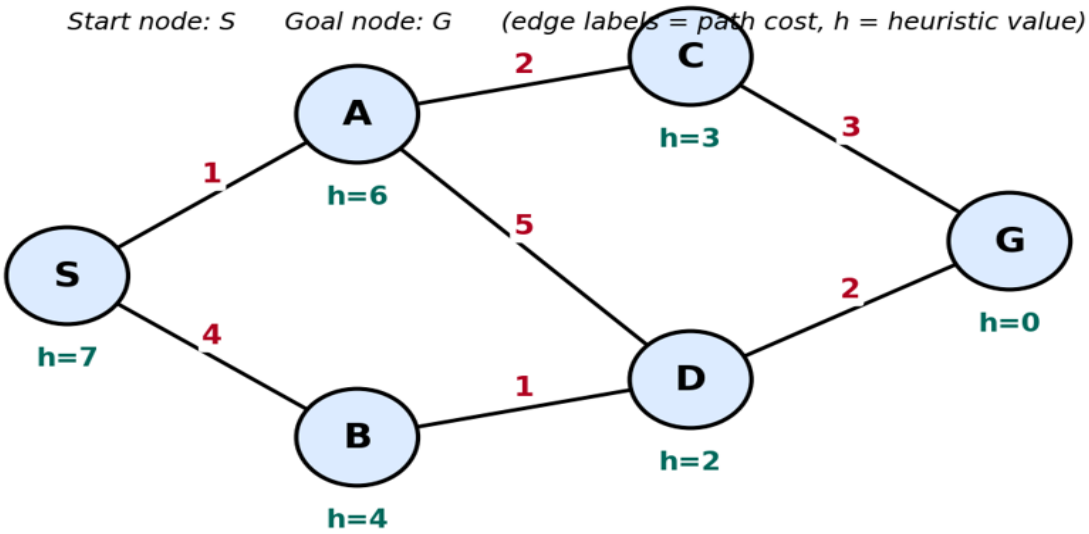
S → B → D → G

Final Path
S → B → D → G

Result
Greedy Best-First Search selects the node having the smallest heuristic value at every step. The algorithm successfully reaches the goal through the path:
S → B → D → G

Q2. A Search Algorithm*

Problem Statement
Consider the weighted graph with heuristic values. Perform a dry run of the A* Search algorithm by computing **g(n)**, **h(n)** and **f(n)=g(n)+h(n)** for every node. Determine the optimal path from the start node **S** to the goal node **G**.



Algorithm (A Search)*

1. Insert the start node into the Open List.
2. Compute
    - g(n): Cost from the start node.
    - h(n): Heuristic estimate.
    - f(n)=g(n)+h(n).
3. Expand the node with the minimum f(n).
4. Update the costs of successor nodes.
5. Repeat until the goal node is reached.

Dry Run
Step 1
Start Node

NODE	g(n)	h(n)	f(n)
S	0	7	7

Expand **S**
Generated nodes:

NODE	g	h	f
A	1	6	7
B	4	4	8

Open List:
A(7), B(8)

Step 2
Expand **A**
Generated successors

NODE	g	h	f
C	3	3	6
D	6	2	8

Open List:
C(6), B(8), D(8)

Step 3
Expand **C**
Generated successor

NODE	g	h	f
G	6	0	6

Open List:
G(6), B(8), D(8)

Step 4
Expand **G**
Goal node reached.

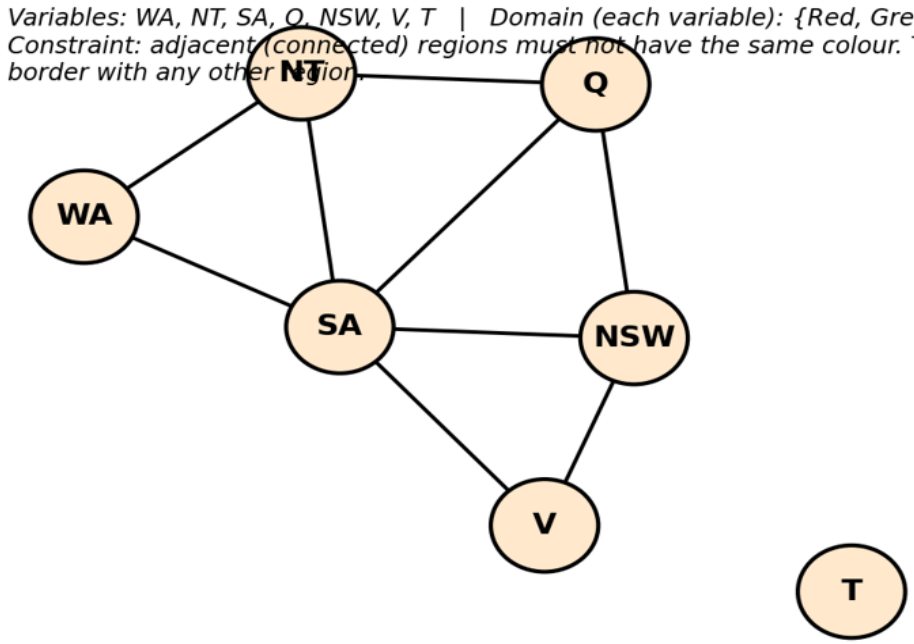
Expansion Order
S → A → C → G

Optimal Path
S → A → C → G

Total Path Cost
= 1 + 2 + 3
= **6**

Result
The A* Search algorithm finds the optimal path
S → A → C → G
with the minimum total cost of **6**.

Q3. Constraint Satisfaction Problem (Map Colouring)
Problem Statement
Formulate the Australian Map Colouring Problem as a Constraint Satisfaction Problem and solve it using Backtracking Search.



Variables
WA, NT, SA, Q, NSW, V and T

Domain
Each variable can take one of the following colours:
{Red, Green, Blue}

Constraints
Adjacent states cannot have the same colour.

- WA ≠ NT
- WA ≠ SA
- NT ≠ SA
- NT ≠ Q
- SA ≠ Q
- SA ≠ NSW
- SA ≠ V
- NSW ≠ V
- Q ≠ NSW

T has no neighbouring state.

Dry Run using Backtracking
Step 1
Assign
WA = Red

Step 2
Assign
NT = Green
Constraint satisfied.

Step 3
Assign
SA = Red
Conflict with WA.
Backtrack.
Assign
SA = Green
Conflict with NT.
Backtrack.
Assign
SA = Blue

Step 4
Assign
Q = Green

Step 5
Assign
NSW = Green
Conflict with Q.
Backtrack.
Assign
NSW = Blue
Conflict with SA.
Backtrack.
Assign
NSW = Red

Step 6
Assign
V = Blue
Conflict with SA.
Backtrack.
Assign
V = Red
Conflict with NSW.
Backtrack.
Assign
V = Green

Step 7
Assign
T = Red
No constraints.

Final Assignment

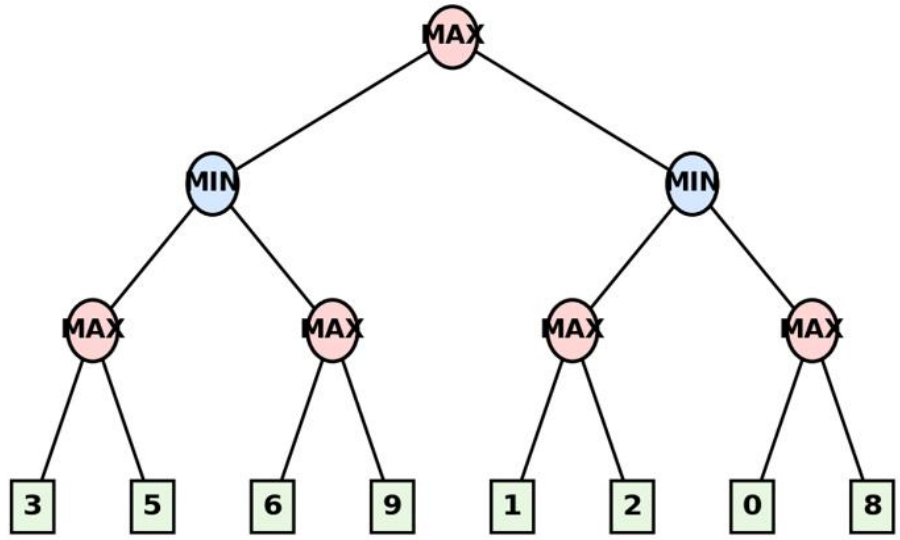
State	Colour
WA	**RED**
NT	**GREEN**
SA	**BLUE**
Q	**GREEN**
NSW	**RED**
V	**GREEN**
T	**RED**

Result
The Backtracking Search algorithm successfully assigns colours to all states without violating any constraints.

Q4. Minimax Algorithm

Problem Statement
Using the given game tree, perform the Minimax algorithm and determine the best move for the MAX player.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Algorithm

1. Assign utility values to all terminal nodes.
2. MAX chooses the maximum value.
3. MIN chooses the minimum value.
4. Continue until the root node is evaluated.

Dry Run
MAX Level

- max(3,5) = 5
- max(6,9) = 9
- max(1,2) = 2
- max(0,8) = 8

MIN Level
Left MIN
min(5,9)=5
Right MIN
min(2,8)=2

Root MAX
max(5,2)=5

Backed-up Values

NODE	**VALUE**
MAX(3,5)	5
MAX(6,9)	9
Left MIN	5
MAX(1,2)	2
MAX(0,8)	8
Right MIN	2
Root MAX	5

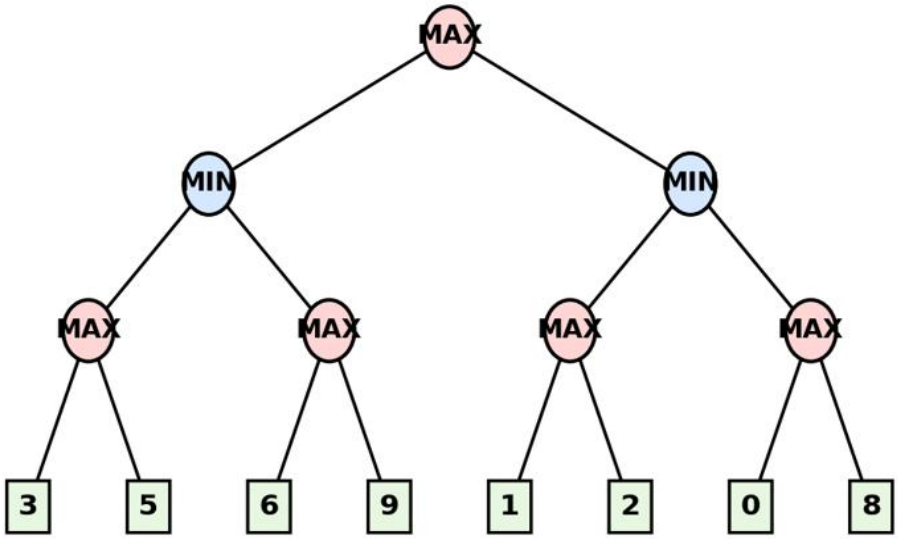
Best Move
Choose the **Left Subtree**
Utility Value = **5**

Result
The Minimax algorithm returns a utility value of **5**. Hence, the best move for the MAX player is the **left branch**.

Q5. Minimax with Alpha-Beta Pruning

Problem Statement
Apply Alpha-Beta Pruning on the given game tree and identify the branches that are pruned while determining the optimal move.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Algorithm
1. Initialize $\alpha = -\infty$ and $\beta = +\infty$.
2. Traverse the tree using Minimax.
3. Update α at MAX nodes and β at MIN nodes.
4. If $\alpha \geq \beta$, prune the remaining branches.
5. Continue until the root node is evaluated.

Dry Run
Initial Values
$\alpha = -\infty$
$\beta = +\infty$

Left MIN Node
Evaluate MAX(3,5)
Value = 5
$\beta = 5$

Evaluate MAX(6,9)
First leaf = 6
$\alpha = 6$
Since
$\alpha \geq \beta$

$6 \geq 5$
The leaf node **9** is pruned.

Left MIN returns
min(5,6)=5
Root updates
$\alpha = 5$

Right MIN Node
Evaluate MAX(1,2)
Value = 2
$\beta = 2$
Now
$\beta \leq \alpha$
$2 \leq 5$
Therefore, the remaining subtree containing **0** and **8** is pruned.

Pruned Nodes
- Leaf node **9**
- Entire subtree containing **0** and **8**

Final Utility
Root = max(5,2)
= **5**

Best Move
Choose the **Left Subtree**

Result
Alpha-Beta Pruning reduces the number of nodes evaluated while producing the same optimal result as the Minimax algorithm. The final utility value is **5**, and the best move for the MAX player is the **left subtree**.